

Project 1: E-commerce Lakehouse

Incremental, governed Databricks lakehouse over Olist e-commerce data

An online retailer exports operational data as CSV files and analysts rebuild spreadsheets by hand. This manual builds an automated, incremental, governed lakehouse with a sales dashboard — repository layout, monthly batch preparation, Bronze/Silver/Gold pipelines, data-quality expectations and quarantine tables, SCD2 dimensions, governance and asset-bundle deployment.

This document contains

- Part B: the full Project 1 build, day by day (manual pages 13-30)
- Part A shared set-up and naming standards (manual pages 3-12)
- Parts F and G: finishing, publishing, troubleshooting and the full command reference (manual pages 59-64)

Vivek Pandey — Data Engineer

Azure · Databricks · PySpark · Delta Lake
vp171097.github.io/DevWebsite

PART A

Standards and Shared Set-up

Do this part once. All projects depend on it.

A1. Naming standards

Object	Rule	Examples
GitHub repos	lowercase, hyphen-separated	databricks-ecommerce-lakehouse
Catalog	one for all portfolio work	portfolio
Schemas	[project]_[layer]; prefixes ecom, perf, cdc	ecom_bronze, perf_gold, cdc_silver
Tables	snake_case; gold uses dim_ / fact_ / agg_	orders, dim_customer, agg_daily_sales
Quarantine tables	[table]_quarantine, same schema	ecom_silver.orders_quarantine
Technical columns	leading underscore	_ingested_at, _source_file, _batch_id
Time columns	_at for timestamps, _date for dates	purchased_at, order_date
Pipelines / jobs	[project]_[purpose]_pipeline / _job	ecom_lakehouse_pipeline, ecom_daily_job
Branches	main + feature/[short-name]	feature/silver-layer
Commits	type: summary (feat, fix, docs, test, refactor, chore, perf)	feat: add SCD2 customer dimension

A2. Accounts to create

Account	Used for	Notes
GitHub	Repos, profile, CI/CD	Turn on two-factor authentication
Databricks Free Edition	Pipelines, jobs, dashboards	Sign up with your professional email
Kaggle	Olist dataset (Project 1)	Account → Settings → API → “Create New Token” downloads kaggle.json
Neon (or Supabase)	Hosted PostgreSQL (Project 3)	Alternative: Docker PostgreSQL locally
draw.io / Excalidraw	Diagrams	Browser, no install
Medium / Hashnode / dev.to	Blog posts	Pick one
Microsoft Azure (free account)	Azure tracks and Project 4	Set a budget alert first (A9)

A3. Install local tools

Step 1: Git, Python 3.11 and Java 17

► Terminal · anywhere

```
# macOS (Homebrew)
brew install git python@3.11 openjdk@17
sudo ln -sfn "$(brew --prefix)/opt/openjdk@17/libexec/openjdk.jdk" \
/Library/Java/JavaVirtualMachines/openjdk-17.jdk

# Ubuntu / WSL
sudo apt update && sudo apt install -y git python3.11 python3.11-venv openjdk-17-jdk

# Windows (PowerShell)
winget install Git.Git Python.Python.3.11 EclipseAdoptium.Temurin.17.JDK
```

► Terminal · verify

```
git --version          # expect 2.40+
python3.11 --version   # expect 3.11.x    (Windows: py -3.11 --version)
java -version          # expect 17.x
```

If PySpark later says it cannot find Java, set JAVA_HOME in your shell profile (~/.zshrc or ~/.bashrc):

► Terminal · anywhere

```
# macOS
echo 'export JAVA_HOME=$(/usr/libexec/java_home -v 17)' >> ~/.zshrc && source ~/.zshrc
# Ubuntu / WSL
echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >> ~/.bashrc && source ~/.bashrc
```

Step 2: Git identity and SSH key for GitHub

► Terminal · anywhere

```
git config --global user.name "[Your Name]"
git config --global user.email "[you@yourbrand.com]"
git config --global init.defaultBranch main

ssh-keygen -t ed25519 -C "[you@yourbrand.com]" # press Enter to accept defaults
cat ~/.ssh/id_ed25519.pub                     # copy the output
# Browser: GitHub → Settings → SSH and GPG keys → New SSH key → paste
# expect: "Hi [username]! You've successfully authenticated"
ssh -T git@github.com
```

Step 3: Databricks CLI

► Terminal · anywhere

```
# macOS / Linux / WSL
curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
# or macOS: brew tap databricks/tap && brew install databricks
# Windows: winget install Databricks.DatabricksCLI

databricks -v # expect v0.2xx or newer
```

Step 4: VS Code and extensions

- Install VS Code, then the extensions: **Python**, **Databricks**, **YAML**, **Ruff**, **GitHub Actions**, **Rainbow CSV**.
- Open a repo folder with code . from its root.

Step 5: optional tools

► Terminal · anywhere

```
brew install --cask docker # Docker Desktop (Project 3 local Postgres alternative)
# psql client (Ubuntu: sudo apt install postgresql-client)
brew install libpq && brew link --force libpq
brew install gh # GitHub CLI (optional, for secrets and PRs from terminal)
```

Step 6: folder for all projects

► Terminal · anywhere

```
mkdir -p ~/portfolio && cd ~/portfolio
```

A4. Connect the CLI to Databricks

- 1 Sign in to Databricks Free Edition in the browser. Copy the workspace URL from the address bar (everything up to and including .com or .net), for example `https://dbc-1234abcd-5678.cloud.databricks.com`.
- 2 Log in from the terminal with a named profile `portfolio`. A browser window opens; approve it.

► Terminal · anywhere

```
databricks auth login --host https://[your-workspace-url] --profile portfolio
databricks auth profiles # 'portfolio' should show Valid: YES
databricks current-user me --profile portfolio
export DATABRICKS_CONFIG_PROFILE=portfolio # add to ~/.zshrc so every command uses it
```

Fallback if browser login is not available: create a personal access token in **Settings → Developer → Access tokens → Generate new token** (lifetime 90 days) and run:

► Terminal · anywhere

```
databricks configure --token --profile portfolio # paste host, then token
```

A5. Create the catalog, schemas and Volumes

Run either the SQL version (in the Databricks SQL editor) or the CLI version (terminal). Both produce the same result.

► SQL editor

```
CREATE CATALOG IF NOT EXISTS portfolio;

CREATE SCHEMA IF NOT EXISTS portfolio.ecom_landing;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_silver;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_ops;

CREATE SCHEMA IF NOT EXISTS portfolio.perf_raw;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_bench;

CREATE SCHEMA IF NOT EXISTS portfolio.cdc_landing;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_silver;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_ops;

CREATE VOLUME IF NOT EXISTS portfolio.ecom_landing.raw;
-- raw files for exploration only
CREATE VOLUME IF NOT EXISTS portfolio.ecom_landing.profiling;
CREATE VOLUME IF NOT EXISTS portfolio.perf_raw.tlc;
CREATE VOLUME IF NOT EXISTS portfolio.cdc_landing.changes;

SHOW SCHEMAS IN portfolio;
SHOW VOLUMES IN portfolio.ecom_landing;
```

► Terminal • alternative to SQL

```
databricks catalogs create portfolio
for s in ecom_landing ecom_bronze ecom_silver ecom_gold ecom_ops \
    perf_raw perf_bronze perf_gold perf_bench \
    cdc_landing cdc_bronze cdc_silver cdc_gold cdc_ops; do
  databricks schemas create "$s" portfolio
done
databricks volumes create portfolio ecom_landing raw MANAGED
databricks volumes create portfolio ecom_landing profiling MANAGED
databricks volumes create portfolio perf_raw tlc MANAGED
databricks volumes create portfolio cdc_landing changes MANAGED
databricks schemas list portfolio
```

WATCH OUT

If Free Edition refuses `CREATE CATALOG`, run `SHOW CATALOGS;`, use the workspace's default catalog instead of `portfolio` everywhere in this manual, and keep all schema names the same. Also note: Free Edition runs on serverless compute with daily quotas and may block outbound internet from notebooks, so every internet-dependent step here runs on your laptop.

A6. Link GitHub to Databricks

- 1 Browser: Databricks → your profile icon → **Settings** → **Linked accounts** → **Git integration** → provider GitHub → link (OAuth app, or a GitHub fine-grained token with repo contents read/write).
- 2 Each time you create a repo: Workspace → your user folder → **Create** → **Git folder** → paste the repo HTTPS URL. Pull changes from the Git folder's branch menu after every local push.

A7. Standard files in every repo

File	Exact contents to put in it
.gitignore	Lines: .env/ __pycache__/* .pyc .env data/ .databricks/* .log .pytest_cache/ .coverage htmlcov/ .DS_Store
.env.example	One setting per line with an empty value: <code>DATABRICKS_CONFIG_PROFILE=portfolio</code> , plus project-specific keys listed in each project
requirements-dev.txt	<code>pyspark==3.5.* · pytest · pytest-cov · chispa · ruff · databricks-sdk · python-dotenv · pandas · pyarrow · requests</code> , plus project extras
pyproject.toml	[project] name and <code>requires-python = ">=3.11"</code> ; [tool.ruff] <code>line-length = 110</code> , <code>target-version py311</code> ; [tool.pytest.ini_options] <code>testpaths = ["tests"]</code> , <code>pythonpath = ["src"]</code>
LICENSE	MIT (choose it when creating the repo on GitHub)
README.md	Structure in Part F

Standard commands you'll use in every repo

► Terminal · repo root

```
# one-time per repo
python3.11 -m venv .venv
source .venv/bin/activate                                # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements-dev.txt                       # then fill in values
cp .env.example .env

# every working session
source .venv/bin/activate
git pull
ruff check . && ruff format --check .                    # lint
pytest -q                                                # tests

# feature workflow
git checkout -b feature/[short-name]
git add -A && git commit -m "feat: [summary]"
git push -u origin feature/[short-name]
# GitHub: open a pull request, merge it, then:
git checkout main && git pull && git branch -d feature/[short-name]
```

A8. The testable-code pattern

All transformation logic lives in plain functions in `src/[package]/transformations/`: each takes a `DataFrame` (plus parameters) and returns a `DataFrame`, with no reading or writing of tables inside. Pipeline files only read, call functions, and declare tables. That's what lets `pytest` run everything locally with a small in-memory `SparkSession`, and it keeps pipeline files short enough for a client to read.

A9. Azure set-up (for the Azure tracks and Project 4)

Every project in this manual has an **Azure track** that swaps free tools for the Azure services clients actually use: **ADLS Gen2**, **Azure SQL Database**, **Cosmos DB**, **Function Apps**, **Virtual Machines**, plus Key Vault. Build each project on Databricks Free Edition first, then do its Azure track in one focused session and delete the resources afterwards. Do this section once, just before your first Azure track.

Azure resource names (used everywhere below)

Resource	Name	Notes
Resource group	rg-dbx-portfolio	Everything lives here; delete it to remove everything
Region	westeurope	Or your nearest region; keep everything in one region
Storage (ADLS Gen2)	stdbxportfolio\$SUFFIX	Containers landing and lake
Access Connector	ac-dbx-portfolio	Managed identity Databricks uses to reach ADLS
Azure Databricks	dbw-portfolio	Trial pricing tier; CLI profile azure
Key Vault	kv-dbx-portfolio-\$SUFFIX	Databricks secret scope kv-portfolio
Azure SQL	sql-dbx-portfolio-\$SUFFIX	Databases shopdb (Project 3) and servingdb (Projects 1 and 4)
Cosmos DB	cosmos-dbx-portfolio-\$SUFFIX	Database appdb; containers customer_profiles, customer_360
Function App	func-dbx-portfolio-\$SUFFIX	Flex Consumption, Python 3.11, system-assigned identity
Virtual Machine	vm-legacy-portfolio	Only in Project 4 (legacy SQL Server + self-hosted runtime)

Resource	Name	Notes
Log Analytics	log-dbx-portfolio	Diagnostic logs (Project 4)

Step 1: variables, login and budget

► Terminal · ~/portfolio/azure.env

```
# put these in ~/portfolio/azure.env and run: source ~/portfolio/azure.env
(SUFFIX = 3-5 random digits)
export SUFFIX=4821
export RG=rg-dbx-portfolio LOC=westeurope
export ST=stdbxportfolio$SUFFIX KV=kv-dbx-portfolio-$SUFFIX
export SQLSRV=sql-dbx-portfolio-$SUFFIX COSMOS=cosmos-dbx-portfolio-$SUFFIX \
  FUNC=func-dbx-portfolio-$SUFFIX
# your Entra object id (after az login)
export ME=$(az ad signed-in-user show --query id -o tsv)
```

► Terminal · repo root

```
az login
az account set --subscription "[subscription name or id]"
source ~/portfolio/azure.env
az group create -n $RG -l $LOC --tags owner=[you] env=lab project=portfolio
az extension add -n databricks
```

Browser: **Cost Management** → **Budgets** → **Add**: scope rg-dbx-portfolio, \$20 per month, alerts at 50/80/100% to your email. Do it before creating anything else.

Step 2: ADLS Gen2, Access Connector and Key Vault

► Terminal · repo root

```
az storage account create -n $ST -g $RG -l $LOC --sku Standard_LRS --kind StorageV2 --hns true \
  --min-tls-version TLS1_2 --allow-blob-public-access false
export ST_ID=$(az storage account show -n $ST -g $RG --query id -o tsv)
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Storage Blob Data Contributor" --scope $ST_ID
az storage fs create -n landing --account-name $ST --auth-mode login
az storage fs create -n lake --account-name $ST --auth-mode login

az databricks access-connector create -n ac-dbx-portfolio -g $RG -l $LOC --identity-type \
  SystemAssigned
export AC_ID=$(az databricks access-connector show -n ac-dbx-portfolio -g $RG --query id -o tsv)
export AC_PID=$(az databricks access-connector show -n ac-dbx-portfolio -g $RG --query \
  identity.principalId -o tsv)
az role assignment create --assignee-object-id $AC_PID --assignee-principal-type \
  ServicePrincipal \
  --role "Storage Blob Data Contributor" --scope $ST_ID

az keyvault create -n $KV -g $RG -l $LOC --enable-rbac-authorization true
export KV_ID=$(az keyvault show -n $KV --query id -o tsv)
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Key Vault Secrets Officer" --scope $KV_ID
# AzureDatabricks first-party application must read secrets for the Key Vault-backed scope
az role assignment create --assignee 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d \
  --role "Key Vault Secrets User" --scope $KV_ID
# note the DNS name for the scope
az keyvault show -n $KV --query properties.vaultUri -o tsv
```

Step 3: Azure Databricks workspace and Unity Catalog

► Terminal · repo root

```
az databricks workspace create -n dbw-portfolio -g $RG -l $LOC --sku trial
export DBW_URL=https://$(az databricks workspace show -n dbw-portfolio -g $RG --query \
  workspaceUrl -o tsv)
databricks auth login --host $DBW_URL --profile azure
databricks current-user me --profile azure
# vCPU quota for clusters
az vm list-usage -l $LOC -o table | grep -i -E "total regional|DDSV5"
```

- 1 Browser, Azure Databricks: **Catalog → External data → Credentials → Create credential** → Azure Managed Identity → Access Connector ID = the value of \$AC_ID → name cred_adls_portfolio.
- 2 Create the Key Vault-backed secret scope: open [DBW_URL]#secrets/createScope, scope name kv-portfolio, DNS name = vault URI, resource ID = \$KV_ID.
- 3 Compute: create a cluster policy or personal compute with **single node, Standard_D4ds_v5, auto-termination 20 minutes**. Free-trial subscriptions have small vCPU quotas.
- 4 Run the SQL below, then the schema and Volume SQL from A5 (skip its CREATE CATALOG line).

► SQL editor · Azure Databricks

```
CREATE EXTERNAL LOCATION IF NOT EXISTS loc_landing
  URL 'abfss://landing@stdbxportfolio[SUFFIX].dfs.core.windows.net/' WITH (STORAGE CREDENTIAL
    cred_adls_portfolio);
CREATE EXTERNAL LOCATION IF NOT EXISTS loc_lake
  URL 'abfss://lake@stdbxportfolio[SUFFIX].dfs.core.windows.net/' WITH (STORAGE CREDENTIAL
    cred_adls_portfolio);
CREATE CATALOG IF NOT EXISTS portfolio
  MANAGED LOCATION 'abfss://lake@stdbxportfolio[SUFFIX].dfs.core.windows.net/portfolio';
SHOW EXTERNAL LOCATIONS;
```

WATCH OUT

Cost discipline: terminate clusters when you stop, keep auto-termination on, deallocate VMs, and when the Azure track is finished run `az group delete -n $RG --yes --no-wait`. Keep screenshots; they are the portfolio evidence, not the running resources. For the next Azure track, repeat Steps 1–3 (about 20 minutes).

Python packages for Azure tracks (add to requirements-dev.txt when needed)

azure-identity (DefaultAzureCredential uses your az login locally and managed identity in Azure) ·
 azure-storage-file-datalake · azure-keyvault-secrets · azure-cosmos · pyodbc (plus the Microsoft ODBC
 Driver 18 for SQL Server) · azure-functions (inside the Function App project only).

A10. Uploading data into Databricks (Volumes and tables)

In Unity Catalog, **files** live in **Volumes** (governed folders such as /Volumes/portfolio/ecom_landing/raw) and **tables** are Delta tables in a schema. The pipelines in this manual read files from Volumes with Auto Loader, so the main task is getting files into the right Volume folder. Pick the method that fits the job:

#	Method	Best for	Used in
1	Catalog Explorer: Upload to this volume (browser)	A few files, quick exploration	Project 1 profiling files
2	Add data → Create or modify table (browser)	One CSV/JSON/Parquet file straight into a table	Quick sandbox tables
3	Databricks CLI databricks fs cp	Folders and larger files from your laptop	All projects
4	Python SDK scripts (upload_batch.py, upload_tlc.py)	Repeatable, logged uploads	Projects 1 and 2
5	SQL : read_files, COPY INTO	Turning Volume files into tables	Exploration, checks
6	Notebook : %sh curl / dbutils.fs.cp	Downloading from the internet inside Databricks	Project 2 (if internet is allowed)

#	Method	Best for	Used in
7	Azure: azcopy to ADLS Gen2 + external Volume	Client-style landing zones	Azure tracks, Project 4

First: create a sandbox schema for exploration tables

► SQL editor

```
CREATE SCHEMA IF NOT EXISTS portfolio.sandbox COMMENT 'Throw-away tables for exploration';
```

Method 1: upload files to a Volume in the browser

- 1 Left sidebar → **Catalog** → portfolio → ecom_landing → **Volumes** → profiling.
- 2 Optional: **Create directory** (top right) to make a sub-folder, for example olist.
- 3 Click **Upload to this volume**, drag the 9 CSV files from data/raw into the dialog, and click **Upload**.
- 4 Check the files are listed in the Volume page, or run the LIST command below.
- 5 Copy a file's path with the **⋮** → **Copy path** menu when you need it in a notebook.

► SQL editor · verify

```
LIST '/Volumes/portfolio/ecom_landing/profiling/';
```

Method 2: create a table directly from a file (browser)

- 1 Left sidebar → **New (+)** → **Add or upload data** → **Create or modify table**.
- 2 Drop one file (for example olist_orders_dataset.csv).
- 3 Choose catalog portfolio, schema sandbox, table name orders_raw.
- 4 Check the preview: first row as header on; for the reviews file open **Advanced attributes** and enable **multi-line** so comments with line breaks stay in one row.
- 5 Adjust column types if needed, then **Create table**. The result is a managed Delta table you can query at once.

TIP

Use Method 2 only for exploration. Pipelines must read files from Volumes, so production-style flows use Methods 3, 4 or 7.

Method 3: Databricks CLI (from your laptop)

► Terminal · repo root

```
# one file
databricks fs cp data/raw/olist_orders_dataset.csv \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv
# a whole folder (recursive)
databricks fs cp -r data/raw dbfs:/Volumes/portfolio/ecom_landing/profiling/
# one batch into the landing folders the pipeline reads
databricks fs cp -r data/batches/2016_09/orders dbfs:/Volumes/portfolio/ecom_landing/raw/orders/
# replace an existing file (never do this in raw landing folders)
databricks fs cp --overwrite data/raw/olist_orders_dataset.csv \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv
# inspect and clean up
databricks fs mkdir dbfs:/Volumes/portfolio/ecom_landing/profiling/olist
databricks fs ls dbfs:/Volumes/portfolio/ecom_landing/profiling/
databricks fs cat \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/product_category_name_translation.csv
databricks fs rm -r dbfs:/Volumes/portfolio/ecom_landing/profiling/olist
```

Method 4: the upload scripts (repeatable and logged)

► Terminal · repo root

```
python scripts/upload_batch.py --batch 0000_initial      # Project 1: reference data
python scripts/upload_batch.py --next                  # Project 1: next monthly batch
python scripts/upload_batch.py --all --dry-run          # show what would be uploaded
# Project 2: TLC Parquet + zone lookup
python scripts/upload_tlc.py
```

Method 5: SQL on files in Volumes

► SQL editor

```
-- peek at files without creating anything
SELECT * FROM read_files('/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv',
                        format => 'csv', header => true) LIMIT 10;

-- create a table from a file (types inferred)
CREATE OR REPLACE TABLE portfolio.sandbox.orders_raw AS
SELECT * FROM read_files('/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv',
                        format => 'csv', header => true, inferSchema => true);

-- multi-line CSV (review comments contain line breaks and quotes)
CREATE OR REPLACE TABLE portfolio.sandbox.reviews_raw AS
SELECT * FROM
  read_files('/Volumes/portfolio/ecom_landing/profiling/olist_order_reviews_dataset.csv',
            format => 'csv', header => true, multiLine => true, escape => '');

-- Parquet folder (Project 2)
SELECT COUNT(*) FROM read_files('/Volumes/portfolio/perf_raw/tlc/yellow/', format => 'parquet');

-- incremental, idempotent load: re-running COPY INTO only loads files it has not loaded before
CREATE TABLE IF NOT EXISTS portfolio.sandbox.orders_copy;
COPY INTO portfolio.sandbox.orders_copy
FROM '/Volumes/portfolio/ecom_landing/raw/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
SELECT COUNT(*) FROM portfolio.sandbox.orders_copy;
```

Method 6: download inside a notebook (only if outbound internet is allowed)

► Notebook cell (%sh)

```
%sh
mkdir -p /Volumes/portfolio/perf_raw/tlc/zones
curl -L -o /Volumes/portfolio/perf_raw/tlc/zones/taxi_zone_lookup.csv \
  https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
ls -lh /Volumes/portfolio/perf_raw/tlc/zones/
```

► Notebook cell (Python)

```
dbutils.fs.ls("/Volumes/portfolio/perf_raw/tlc/zones/")
dbutils.fs.cp("/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv",
              "/Volumes/portfolio/ecom_landing/profiling/copy/orders.csv") # copy a file
```

Method 7: Azure landing zone (Azure tracks)

Upload with azcopy or the Data Lake SDK to the ADLS container, then expose it through an external Volume (commands in A9, B13 and C8). Databricks then reads it exactly like any other Volume path.

WATCH OUT

Files in the **raw landing** Volumes are immutable: Auto Loader tracks what it has processed and ignores a re-uploaded file with the same name. Upload new data as new files, and keep exploration files in the **profiling** Volume or the sandbox schema. UI uploads are meant for small files; use the CLI or scripts for large ones, and check the current upload limits in the Databricks documentation.

Part A checklist

- ☐ Can upload files to a Volume (A10) and create a table from them
- ☐ Tools installed and verified
- ☐ Git identity and SSH key working
- ☐ CLI profile 'portfolio' valid
- ☐ Catalog, 14 schemas and 3 Volumes exist
- ☐ GitHub linked to Databricks

PART B

Project 1: E-commerce Lakehouse

Client story: an online retailer exports its operational data as CSV files; analysts rebuild spreadsheets by hand and numbers never match. You'll deliver an automated, incremental, governed lakehouse with a sales dashboard. **Duration:** about 15 working days.

B1. Repository layout

```
databricks-ecommerce-lakehouse/
├── README.md
├── databricks.yml
├── resources/
│   ├── ecom_lakehouse_pipeline.yml
│   └── ecom_daily_job.yml
├── src/ecom/
│   ├── __init__.py
│   ├── config.py
│   ├── transformations/
│   │   ├── __init__.py
│   │   ├── cleaning.py
│   │   ├── orders.py
│   │   ├── order_items.py
│   │   ├── payments_reviews.py
│   │   ├── customers.py
│   │   ├── products_sellers.py
│   │   └── quality_rules.py
│   └── pipelines/
│       ├── bronze.py
│       ├── silver.py
│       └── gold.py
├── notebooks/
│   ├── 01_explore_raw_data
│   ├── 02_verify_pipeline_run
│   └── 03_data_quality_report
├── scripts/
│   ├── prepare_batches.py
│   ├── generate_customer_updates.py
│   ├── make_bad_rows_batch.py
│   └── upload_batch.py
├── sql/
│   ├── grants.sql
│   └── tags_and_comments.sql
├── tests/
│   ├── conftest.py
│   ├── test_cleaning.py
│   ├── test_orders.py
│   ├── test_customers.py
│   └── test_quality_rules.py
├── docs/ architecture.png data_model.png governance.md screenshots/
├── data/ # git-ignored
├── .env.example .gitignore pyproject.toml requirements-dev.txt LICENSE
```

B2. Build schedule

Day	Work	Commit message at the end of the day
1	Repo, venv, config, download data, profiling	chore: project skeleton and config
2	Batch, update, bad-row and upload scripts	feat: batch preparation and upload scripts
3-4	Bronze layer, first pipeline runs	feat: bronze ingestion with Auto Loader
5-7	Silver layer, rules, quarantine, unit tests	feat: silver layer with expectations
8-10	Gold star schema, SCD2, aggregates	feat: gold star schema and aggregates

Day	Work	Commit message at the end of the day
11	Job, trigger, alerts, check and DQ notebooks	feat: orchestration and data quality report
12	Governance	docs: governance and lineage
13	Dashboard	feat: sales dashboard
14	Asset Bundle	feat: asset bundle deployment
15	Full run-through, README, diagrams, blog	docs: README and architecture

B3. Day 1: skeleton, config and data

Create the repo and skeleton

- 1 Browser: GitHub → New repository → name databricks-e-commerce-lakehouse → Public → add README, .gitignore template Python, licence MIT → Create.
- 2 Clone, create the structure, set up the environment and push (commands below).
- 3 Browser: Databricks → Workspace → your user folder → Create → Git folder → paste the repo URL.

► Terminal · repo root

```
cd ~/portfolio
git clone git@github.com:[your-github-username]/databricks-e-commerce-lakehouse.git
cd databricks-e-commerce-lakehouse

mkdir -p resources src/ecom/transformations src/ecom/pipelines notebooks scripts sql tests \
      docs/screenshots data/raw data/batches
touch src/ecom/__init__.py src/ecom/config.py src/ecom/transformations/__init__.py \
      src/ecom/transformations/{cleaning,orders,order_items,payments_reviews}.py \
      src/ecom/transformations/{customers,products_sellers,quality_rules}.py \
      src/ecom/pipelines/{bronze,silver,gold}.py \
      scripts/{prepare_batches,generate_customer_updates,make_bad_rows_batch,upload_batch}.py \
      sql/grants.sql sql/tags_and_comments.sql \
      tests/{conftest,test_cleaning,test_orders,test_customers,test_quality_rules}.py \
      databricks.yml resources/ecom_lakehouse_pipeline.yml resources/ecom_daily_job.yml \
      .env.example requirements-dev.txt pyproject.toml docs/governance.md
# now fill .gitignore, requirements-dev.txt, pyproject.toml per A7 and add: kaggle

python3.11 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip && pip install -r requirements-dev.txt
git add -A && git commit -m "chore: project skeleton and config" && git push
```

Download the Olist dataset

► Terminal · repo root

```
mkdir -p ~/.kaggle && mv ~/Downloads/kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
kaggle datasets download -d olistbr/brazilian-e-commerce -p data/raw --unzip
ls -lh data/raw # expect the 9 CSV files below
# rough row counts (header + rows; reviews has multi-line text)
wc -l data/raw/*.csv
```

Kaggle file	Source name	Business key	Rows
olist_orders_dataset.csv	orders	order_id	99,441
olist_order_items_dataset.csv	order_items	order_id + order_item_id	112,650
olist_order_payments_dataset.csv	order_payments	order_id + payment_sequential	103,886
olist_order_reviews_dataset.csv	order_reviews	review_id + order_id	99,224
olist_customers_dataset.csv	customers	customer_id (person: customer_unique_id)	99,441

Kaggle file	Source name	Business key	Rows
olist_products_dataset.csv	products	product_id	32,951
olist_sellers_dataset.csv	sellers	seller_id	3,095
olist_geolocation_dataset.csv	geolocation	zip prefix (many rows each)	1,000,163
product_category_name_translation.csv	category_translation	product_category_name	71

src/ecom/config.py

Purpose	One place for every name and path. Nothing else in the repo hard-codes names.
Contains	- CATALOG = "portfolio" - SCHEMAS: landing = ecom_landing, bronze = ecom_bronze, silver = ecom_silver, gold = ecom_gold, ops = ecom_ops - LANDING_PATH = /Volumes/portfolio/ecom_landing/raw (overridable by the pipeline configuration key ecom.landing_path) - SOURCE_FILES: dictionary from source name to Kaggle file name (the table above) - SOURCES: the 9 source names plus customer_updates (10 in total) - KEYS: dictionary from source name to list of key columns (the table above) - ORDER_STATUSES: delivered, shipped, canceled, unavailable, invoiced, processing, created, approved - Function fq(layer, table) returning "portfolio.[schema of layer].[table]" - Function landing(source) returning LANDING_PATH + "/" + source
Done when	Searching the repo for the text "portfolio." finds matches only in config.py and the SQL files.

notebooks/01_explore_raw_data

Purpose	Profile the raw data before designing anything. The findings become README material.
Runs where	Databricks notebook (Python). First upload the 9 raw CSV files to the profiling Volume (A10, method 1 or 2). Never put exploration files in the raw Volume: Auto Loader would ingest them.
Logic	- For each file: read with header on and everything as string; print row count and column list. - Nulls: for every column count nulls and empty strings; show as a table sorted by null percentage. - Keys: count distinct values of the business key and compare to row count; list sample duplicate keys. - Dates (orders): min and max of each timestamp column; count values that fail to parse. - Numbers (order_items, payments): min, max, average, count of values at or below 0. - Categoricals: distinct values and counts for order_status, payment_type, customer_state, review_score. - Relationships: count order_items whose order_id is missing from orders; orders whose customer_id is missing from customers. - Reviews: count rows where the comment contains a line break (explains the multi-line setting later). - Write a Markdown cell "Findings" with 6-10 bullet points (for example duplicate review_ids exist; product columns are misspelled "lenght"; geolocation has many rows per zip prefix).
Done when	Findings cell written and copied into the README draft.

B4. Day 2: batch preparation, updates, bad rows and upload

Real clients receive data in drops, not one big file. You'll split Olist by **order purchase month** (September 2016 to October 2018: 25 months, with a few very small months at the edges) so your pipeline visibly processes data incrementally.

scripts/prepare_batches.py	
Purpose	Split the Olist files into monthly batch folders that simulate exports.
Runs where	Laptop, with pandas (small enough to fit in memory).
Arguments	--raw data/raw --out data/batches --months N (optional: only first N months) --force (delete and recreate the output folder)
Outputs	data/batches/[batch_id]/[source]/[source]_[batch_id].csv and data/batches/batch_manifest.csv
Logic	1. Parse arguments. If the output folder exists and --force isn't given, stop with a clear message. 2. Read every file with all columns as strings (keeps raw values exactly as they are; empty cells stay empty). For reviews, the CSV reader must handle quoted multi-line text. 3. Batch 0000_initial: copy products, sellers, geolocation, category_translation in full. 4. Parse order_purchase_timestamp into a temporary column and derive batch_id = year_month in the form YYYY_MM. Count orders whose timestamp fails to parse; stop if any (there should be none). 5. Sort the distinct batch_ids ascending. If --months is given, keep only the first N. 6. For each batch_id: orders subset = orders in that month (drop the temporary columns before writing); keys = its order_ids and customer_ids. 7. order_items, order_payments, order_reviews subsets = rows whose order_id is in keys. 8. customers subset = rows whose customer_id is in keys (each Olist customer_id belongs to exactly one order). 9. Write each non-empty subset to its path with a header row, UTF-8, the same column order as the source file, and all fields quoted if they contain commas, quotes or line breaks. 10. Append a manifest record per file: batch_id, source, relative path, row count, file size bytes. 11. After the loop, sum row counts per source from the manifest and compare with the original counts. Print a table source / original / batched / difference. Exit with an error code if any difference is non-zero for orders, order_items, payments, reviews or customers.
Done when	The reconciliation table shows difference 0 for every source; about 26 batch folders exist.

► Terminal · repo root

```
python scripts/prepare_batches.py --raw data/raw --out data/batches
# 0000_initial, 2016_09, 2016_10, 2016_12, 2017_01 ...
ls data/batches | head
column -s, -t data/batches/batch_manifest.csv | head -20
```

scripts/generate_customer_updates.py	
Purpose	Create “customer moved” change records so the SCD Type 2 dimension has real history.
Runs where	Laptop, pandas.
Arguments	--batch YYYY_MM (target batch folder) --pct 2 (percent of customers) --seed 42
Outputs	data/batches/[batch]/customer_updates/customer_updates_[batch].csv, plus a manifest record

Logic	1. Read the customers file. Build the list of distinct customer_unique_id values whose first order is before the target batch (read orders to find the first purchase month per customer_unique_id). Customers can't move before they exist. 2. Draw a random sample of pct percent of them using the seed (repeatable output). 3. Build a pool of distinct real (zip_code_prefix, city, state) combinations from customers. 4. For each sampled customer pick a combination from the pool that differs from their current city. 5. change_ts = a random timestamp inside the target month (seeded), formatted like the source timestamps. 6. Write rows with columns: customer_unique_id, customer_zip_code_prefix, customer_city, customer_state, change_ts. 7. Append the manifest record and print how many updates were written.
Done when	Three files exist (for three batches) and at least some customer_unique_ids appear in two of them.

► Terminal · repo root

```
python scripts/generate_customer_updates.py --batch 2017_06 --pct 2 --seed 42
python scripts/generate_customer_updates.py --batch 2017_12 --pct 2 --seed 43
python scripts/generate_customer_updates.py --batch 2018_05 --pct 2 --seed 44
```

scripts/make_bad_rows_batch.py

Purpose	Create a batch containing deliberately broken rows to prove the quality rules and quarantine work.
Arguments	--from-batch 2018_08 --out-batch 9999_bad_rows
Logic	1. Copy 20 orders, their items and reviews from the source batch. 2. Break them on purpose: blank order_id on 3 orders; unknown status "lost" on 3 orders; unparseable purchase timestamp on 2; negative price on 3 items; freight -1 on 2 items; review_score 7 on 3 reviews. 3. Give the broken orders new order_ids (prefix bad_) so they don't collide with real ones. 4. Write to the out-batch folder with the usual naming and add manifest records. 5. Print the expected number of quarantined rows per table (you'll compare with the pipeline).
Done when	Expected quarantine counts printed; files exist in data/batches/9999_bad_rows.

► Terminal · repo root

```
python scripts/make_bad_rows_batch.py --from-batch 2018_08 --out-batch 9999_bad_rows
```

scripts/upload_batch.py

Purpose	Simulate the arrival of an export by copying a batch into the landing Volume.
Runs where	Laptop, with the Databricks SDK (WorkspaceClient reads the 'portfolio' CLI profile) Files API.
Arguments	--batch [batch_id] --next (next batch not yet uploaded) --all --dry-run --profile portfolio
Outputs	/Volumes/portfolio/ecom_landing/raw/[source]/[file] and a line per file in data/upload_log.csv
Logic	1. Read the manifest and data/upload_log.csv (create it if missing) to know what is already uploaded. 2. Resolve which batches to upload: the named one; the next not-yet-uploaded in sorted order; or all remaining in order. 3. For each file: build the Volume path; if it already exists remotely (check with the Files API metadata call) or is in the log, skip it and report "skipped (immutable)". 4. Otherwise upload it (create the source folder first if needed); on failure retry up to 3 times with 2, 4, 8 seconds waits. 5. Append to the log: batch_id, source, file, remote path, bytes, uploaded_at (UTC). 6. With --all, pause 5 seconds between batches so file-arrival triggers see separate drops. 7. With --dry-run, print what would be uploaded and do nothing. 8. End with a summary: uploaded, skipped, failed (non-zero exit code if any failed).

Done when	Uploading the same batch twice uploads nothing the second time.
------------------	---

► Terminal · repo root

```
python scripts/upload_batch.py --batch 0000_initial
python scripts/upload_batch.py --batch 2016_09
databricks fs ls dbfs:/Volumes/portfolio/ecom_landing/raw/           # one folder per source
databricks fs ls dbfs:/Volumes/portfolio/ecom_landing/raw/orders/

# equivalent manual upload with the CLI, if you want to test without the script:
databricks fs cp data/batches/2016_09/orders/orders_2016_09.csv \
  dbfs:/Volumes/portfolio/ecom_landing/raw/orders/orders_2016_09.csv
```

► Terminal · repo root

```
git add -A && git commit -m "feat: batch preparation and upload scripts" && git push
```

B5. Days 3-4: bronze layer

Bronze tables (schema ecom_bronze)

Table	Type	Source folder	Columns
orders, order_items, order_payments, customers, customer_updates, products, sellers, geolocation, category_translation	Streaming tables	raw/[same name]/	All source columns as strings + technical columns
order_reviews	Streaming table	raw/order_reviews/	Same, read with multi-line CSV parsing

Technical column	How it's filled
_ingested_at	Current timestamp at processing time
_source_file	The file path from the hidden file-metadata column
_batch_id	Regular expression on the file name: the part matching 4 digits, underscore, 2 digits (or 0000_initial / 9999_bad_rows)
_rescued_data	Created by Auto Loader; holds values that didn't match the schema. Should stay null.

src/ecom/pipelines/bronze.py	
Purpose	Declare one incremental Auto Loader streaming table per source.
Runs where	Inside the pipeline only (never run it directly; the pipeline engine executes it).
Logic	1. Import the pipelines module (from pyspark import pipelines), functions, and config. 2. Read the landing path from the pipeline configuration key ecom.landing_path, falling back to config.LANDING_PATH. 3. Write a factory function make_bronze(source) that declares a table named fq("bronze", source) with a comment "Raw [source] files from landing, loaded incrementally" and table property quality = bronze. Inside, it returns a streaming read. 4. Streaming read options: cloudFiles.format = csv; header = true; cloudFiles.inferColumnTypes = false; cloudFiles.schemaEvolutionMode = rescue; and for order_reviews only: multiLine = true, quote and escape = double-quote. 5. Load from landing(source) + "/" 6. Add the four technical columns described above. 7. Call make_bronze for every name in config.SOURCES (a loop, not 10 copies).
Done when	First run creates 10 tables whose counts equal the uploaded files' row counts.

Create the pipeline (UI, first time)

- 1 Browser: **Jobs and Pipelines** → **Create** → **ETL pipeline**. Name `ecom_lakehouse_pipeline`.
- 2 Default catalog `portfolio`, default schema `ecom_bronze`.
- 3 Source code: add the Git-folder paths of `src/ecom/pipelines/bronze.py` (add `silver.py` and `gold.py` when they exist).
- 4 Add `src` to the Python path: in pipeline settings, set the root folder to the repo's `src` folder (or add it as a library path), so `import ecom.config` works.
- 5 Compute: **Serverless**. Pipeline mode: **Triggered**. Channel: `Current`.
- 6 Advanced → Configuration: key `ecom.landing_path`, value `/Volumes/portfolio/ecom_landing/raw`.
- 7 Save and click **Start**. Open the graph; click each table to see rows written.

► Terminal · optional

```
databricks pipelines list-pipelines --filter "name LIKE 'ecom%'"      # note the pipeline_id
# start a run from the terminal
databricks pipelines start-update [pipeline_id]
# state and latest updates
databricks pipelines get [pipeline_id]
```

► SQL editor · verify

```
-- Row counts per table and batch (compare with data/batches/batch_manifest.csv)
SELECT _batch_id, COUNT(*) AS rows FROM portfolio.ecom_bronze.orders GROUP BY _batch_id ORDER
  BY 1;
SELECT COUNT(*) FROM portfolio.ecom_bronze.order_reviews;
-- expect 0
SELECT COUNT(*) AS rescued FROM portfolio.ecom_bronze.orders WHERE _rescued_data IS NOT NULL;
-- one STREAMING UPDATE entry per pipeline update
DESCRIBE HISTORY portfolio.ecom_bronze.orders;
```

Prove incremental loading

► Terminal · repo root

```
python scripts/upload_batch.py --next          # uploads 2016_10
databricks pipelines start-update [pipeline_id]
```

In the pipeline's run view, the new update shows only the new rows processed per table. Rerun the `_batch_id` count query: a new batch row appears and old counts are unchanged. Screenshot both runs.

► Terminal · repo root

```
git add -A && git commit -m "feat: bronze ingestion with Auto Loader" && git push
```

B6. Days 5-7: silver layer

Silver tables (schema `ecom_silver`)

Table	Key	Columns (type)	Detailed rules
orders	order_id	order_id, customer_id, order_status, purchased_at, approved_at, delivered_carrier_at, delivered_customer_at, estimated_delivery_at (timestamp), purchase_date (date), is_delivered (boolean), delivery_days (int), delivered_late (boolean), _batch_id, _ingested_at	Trim all strings; empty → null. order_status lower-cased. Parse timestamps with format yyyy-MM-dd HH:mm:ss (unparseable → null). purchase_date = date of purchased_at. is_delivered = status is delivered. delivery_days = whole days from purchased_at to delivered_customer_at (null if not delivered). delivered_late = delivered_customer_at date after estimated_delivery_at date (null if not delivered).

Table	Key	Columns (type)	Detailed rules
order_items	order_id + order_item_id	order_id, order_item_id (int), product_id, seller_id, shipping_limit_at (timestamp), price, freight_value, item_total (decimal 12,2)	Cast numbers; item_total = price + freight_value.
order_payments	order_id + payment_sequential	order_id, payment_sequential (int), payment_type, payment_installments (int), payment_value (decimal 12,2)	payment_type lower-cased; not_defined → unknown; installments 0 → 1.
order_reviews	review_id + order_id	review_id, order_id, review_score (int), review_title, review_message, created_at, answered_at	Trim text; empty → null; line breaks in text replaced with a space. If a key repeats, keep the row with the latest answered_at.
customers	customer_id	customer_id, customer_unique_id, zip_code_prefix (string, 5 chars, left-padded with zeros), city, state	city: trimmed, title case; state: upper case; zip padded to 5 characters.
customer_changes	customer_unique_id + change_ts	customer_unique_id, zip_code_prefix, city, state, change_ts (timestamp), change_source	Union of (a) customers joined to orders: change_ts = purchased_at of that order, change_source = order; (b) customer_updates: change_source = update. Same cleaning as customers.
products	product_id	product_id, category_name_pt, category_name_en, name_length, description_length, photos_qty (int), weight_g, length_cm, height_cm, width_cm (double)	Rename the misspelled source columns (length → length). Join category_translation on the Portuguese name. Missing category → uncategorized (both languages).
sellers	seller_id	seller_id, zip_code_prefix, city, state	Same normalization as customers.
geolocation	zip_code_prefix	zip_code_prefix, lat, lng (double), city, state, source_rows (int)	Materialized view: group by padded zip prefix; lat and lng = averages (only values inside Brazil's bounding box: lat between -34 and 6, lng between -74 and -34); city and state = most frequent value.
category_translation	category_name_pt	category_name_pt, category_name_en	Trim; lower case.

Data quality rules (quality_rules.py)

Table	Rule name	Condition that must be TRUE	Action
orders	valid_order_id	order_id is not null	drop → quarantine
orders	valid_customer_id	customer_id is not null	drop → quarantine
orders	valid_status	order_status is in ORDER_STATUSES	drop → quarantine
orders	valid_purchase_ts	purchased_at is not null	drop → quarantine
orders	delivery_after_purchase	delivered_customer_at is null OR delivered_customer_at at or after purchased_at	warn
order_items	valid_keys	order_id, order_item_id and product_id are not null	drop → quarantine
order_items	positive_price	price greater than 0	drop → quarantine

Table	Rule name	Condition that must be TRUE	Action
order_items	non_negative_freight	freight_value 0 or more	drop → quarantine
order_payments	non_negative_payments	payment_value 0 or more	warn
order_reviews	valid_score	review_score between 1 and 5	drop → quarantine
customers	valid_state	state has exactly 2 letters	warn
products	known_category	category_name_pt is not null in the source	warn

src/ecom/transformations/*.py

Purpose	Pure functions holding all silver logic (unit-testable locally).
cleaning.py	- trim_all_strings(df): for every string column, trim it; return the same column order. - empty_to_null(df): string columns that are empty after trimming become null. - title_case(col) / upper_case(col): return transformed column expressions. - pad_zip(col): left-pad to 5 characters with zeros. - parse_ts(col, fmt): parse with the given format; failures become null (never raise).
orders.py	type_orders(df): applies trim and empty_to_null, lower-cases status, parses the 5 timestamps, renames to the silver column names, derives purchase_date, is_delivered, delivery_days, delivered_late.
order_items.py	type_order_items(df): casts, computes item_total, keeps technical columns.
payments_reviews.py	type_payments(df) and type_reviews(df) as per the rules table.
customers.py	type_customers(df); build_customer_changes(customers_df, orders_df, updates_df) returning the union described above.
products_sellers.py	type_products(products_df, translation_df); type_sellers(df); aggregate_geolocation(df).
quality_rules.py	- RULES: a dictionary per table: {rule_name: condition string} for drop rules, and a separate WARN_RULES dictionary. - quarantine_filter(rules): builds one condition "NOT (rule1) OR NOT (rule2) ...", treating a null result as a failure. - failed_rules_column(rules): builds an array column listing the names of the rules each row fails.
Done when	silver.py contains no business rules of its own, only calls to these functions.

src/ecom/pipelines/silver.py

Purpose	Declare silver tables, expectations and quarantine tables.
Logic	- For each streaming entity (orders, order_items, order_payments, order_reviews, customers): declare a table fq("silver", name) with drop expectations = RULES[name] and warn expectations = WARN_RULES[name]. - Body: streaming read of the bronze table → call its type_function → set a watermark of 1 day on _ingested_at → drop duplicates within the watermark on the business key. - For each entity that has drop rules, declare fq("silver", name + "_quarantine"): streaming read of bronze → same type_function → filter with quarantine_filter → add failed_rules and quarantined_at. - Declare customer_changes as a streaming table built from build_customer_changes. Because it combines customers with orders, read customers and customer_updates as streams and orders as a static read (stream-static join). - products, sellers, category_translation, geolocation: declare as materialized views (full recompute each run; they're small reference tables). Explain this choice in the README. - Every table gets a comment and property quality = silver.
Done when	For each table with drop rules: silver count + quarantine count = bronze count.

► SQL editor · verify

```
-- reconciliation: must return 0 for every row
SELECT 'orders' t, (SELECT COUNT(*) FROM portfolio.ecom_bronze.orders)
  - (SELECT COUNT(*) FROM portfolio.ecom_silver.orders)
  - (SELECT COUNT(*) FROM portfolio.ecom_silver.orders_quarantine) AS diff
UNION ALL
SELECT 'order_items', (SELECT COUNT(*) FROM portfolio.ecom_bronze.order_items)
  - (SELECT COUNT(*) FROM portfolio.ecom_silver.order_items)
  - (SELECT COUNT(*) FROM portfolio.ecom_silver.order_items_quarantine);

-- duplicates on business key: expect no rows
SELECT order_id, COUNT(*) FROM portfolio.ecom_silver.orders GROUP BY order_id HAVING COUNT(*)
  > 1;
```

Unit tests (write them alongside the functions)

File	Test	Arrange → expected result
test_cleaning.py	trim_all_strings	" a ", 1 → "a", 1 (numbers untouched)
	empty_to_null	"" , " " , "x" → null, null, "x"
	parse_ts bad value	"2017-13-45 99:00:00" → null, no error
	pad_zip	"1037" → "01037"
test_orders.py	delivery_days	purchased 2017-01-01 10:00, delivered 2017-01-05 09:00 → 3 (whole days)
	delivered_late	delivered after estimate → true; before → false; not delivered → null
	status lower-cased	"Delivered" → "delivered"
test_customers.py	build_customer_changes	1 customer + 1 order + 1 update → 2 rows, ordered by change_ts, sources order then update
	normalization	"sao paulo", "sp", "1037" → "Sao Paulo", "SP", "01037"
test_quality_rules.py	quarantine_filter	row failing 2 rules selected once; failed_rules has both names; a null order_id counts as a failure

tests/conftest.py

Logic

1. Session-scoped fixture named spark: local master with 1 core, 1 shuffle partition, UI disabled, time zone UTC; stop it after the session.
2. Helper fixture that builds a DataFrame from a list of dicts, for readable tests.

► Terminal · repo root

```
pytest -q                                # all tests
pytest tests/test_orders.py -q           # one file
pytest -q -k delivered_late              # one test by name
ruff check . && ruff format .
git add -A && git commit -m "feat: silver layer with expectations" && git push
```

TIP

Upload the bad rows now to see quarantine working: `python scripts/upload_batch.py --batch 9999_bad_rows`, run the pipeline, then compare quarantine counts with what `make_bad_rows_batch.py` printed. Screenshot it.

B7. Days 8-10: gold layer

Gold tables (schema ecom_gold)

Table	Type / grain	Columns	Detailed build logic
dim_customer	AUTO CDC SCD2 streaming table; one row per customer version	customer_unique_id, zip_code_prefix, city, state, change_source, __START_AT, __END_AT	Source silver.customer_changes; keys customer_unique_id; sequence by change_ts; SCD type 2; track history on zip_code_prefix, city, state (other columns update in place). Current row = __END_AT is null.
dim_product	Materialized view; per product	product_id, category_name_en, category_name_pt, weight_g, volume_cm3, size_band	volume_cm3 = length × height × width; size_band small if volume under 2,000, medium under 20,000, else large; null dims → unknown
dim_seller	MV; per seller	seller_id, city, state, lat, lng	silver.sellers left join silver.geolocation on zip prefix
dim_date	MV; per day 2016-01-01 to 2018-12-31	date, year, quarter, month, month_name, year_month, week_of_year, day_of_week, is_weekend	Generate the date sequence between the two dates, explode to rows, derive attributes
fact_order_item	MV; per order item	order_id, order_item_id, customer_unique_id, product_id, seller_id, order_date, order_status, price, freight_value, item_total, delivery_days, delivered_late, review_score	silver.order_items inner join silver.orders on order_id; join silver.customers on customer_id to get customer_unique_id; left join reviews aggregated to one score per order (average, rounded); order_date = purchase_date
fact_payment	MV; per payment	order_id, payment_sequential, order_date, payment_type, installments, payment_value	silver.order_payments join silver.orders
agg_daily_sales	MV; per day	order_date, orders, items, revenue, freight, avg_order_value, late_delivery_rate	fact_order_item where order_status = delivered, grouped by order_date: orders = distinct order_id; revenue = sum price; avg_order_value = revenue / orders; late_delivery_rate = late orders / orders
agg_category_monthly	MV; month × category	year_month, category_name_en, orders, revenue, avg_review_score	fact_order_item join dim_product join dim_date
agg_seller_performance	MV; per seller	seller_id, state, orders, revenue, avg_delivery_days, late_rate, avg_review_score, rank_by_revenue	fact_order_item join dim_seller; rank with a window over revenue
agg_customer_cohorts	MV; cohort month × months since	cohort_month, months_since_first, active_customers, cohort_size, retention_rate	Per customer: first order month = cohort. Per customer and order month: months_since = month difference. Count distinct customers per (cohort, months_since); divide by the cohort size at months_since = 0

src/ecom/pipelines/gold.py

Logic	1. Declare the streaming table dim_customer (comment, property quality = gold). 2. Declare an AUTO CDC flow into it from silver.customer_changes with the settings in the table above. 3. Declare each materialized view from the catalogue as a read of silver or gold tables by fully qualified name. 4. Money: round to 2 decimals; rates: round to 4. Never turn a missing value into 0 (orders without reviews keep review_score null). 5. Add a table comment and column comments on every gold table (the dashboard and Catalog Explorer show them).
Done when	The three verification queries below return the expected results.

► SQL editor · verify

```
-- 1. Gold revenue equals silver revenue for delivered orders (difference ~ 0)
SELECT (SELECT ROUND(SUM(revenue),2) FROM portfolio.ecom_gold.agg_daily_sales)
      - (SELECT ROUND(SUM(i.price),2) FROM portfolio.ecom_silver.order_items i
        JOIN portfolio.ecom_silver.orders o USING (order_id) WHERE o.order_status =
          'delivered') AS diff;

-- 2. SCD2: customers with more than one version
SELECT customer_unique_id, COUNT(*) versions FROM portfolio.ecom_gold.dim_customer
GROUP BY 1 HAVING COUNT(*) > 1 ORDER BY versions DESC LIMIT 10;

-- 3. One customer's history
SELECT * FROM portfolio.ecom_gold.dim_customer WHERE customer_unique_id = '[id from query 2]'
ORDER BY __START_AT;
```

► Terminal · repo root

```
git add -A && git commit -m "feat: gold star schema and aggregates" && git push
```

B8. Day 11: orchestration, checks and data quality report**notebooks/02_verify_pipeline_run**

Purpose	Automatic checks after every run. The job fails loudly if the data is wrong.
Logic	1. Create an empty list of results. Each check appends (check_name, passed, detail). 2. Reconciliation: for orders, order_items, order_reviews: bronze = silver + quarantine. 3. Uniqueness: no duplicate business keys in any silver table; no duplicate (customer_unique_id, __START_AT) in dim_customer. 4. One current row per customer: count of customers with more than one row where __END_AT is null = 0. 5. Referential integrity: product_ids in fact_order_item missing from dim_product = 0; same for seller_id. 6. Freshness: max(__ingested_at) in bronze.orders is within the last 24 hours. 7. Totals: gold revenue minus silver delivered revenue has an absolute value under 0.01. 8. Write all results with a run timestamp and the job run id (taken from a job parameter {{job.run_id}}) to ecom_ops.run_checks (create it on first use). 9. Print a results table. If any check failed, raise an exception listing the failed checks (this fails the task and triggers the alert).
Done when	A normal run passes; temporarily deleting one row from a dimension (then re-running the pipeline to repair) makes it fail.

notebooks/03_data_quality_report

Purpose	Turn expectation metrics into a chartable table.
Logic	1. Read the pipeline event log with the event_log table-valued function using the pipeline id (passed as a job parameter), or the event-log table if you enabled publishing in pipeline settings. 2. Filter event_type = flow_progress with a non-null data-quality section in the details JSON. 3. Explode the expectations array: dataset, name, passed_records, failed_records; keep origin.update_id and timestamp. 4. Aggregate per update_id, dataset, name; compute failed_pct = failed / (passed + failed). 5. MERGE into ecom_ops.dq_summary on (update_id, dataset, rule_name) so re-runs don't duplicate rows.
Done when	ecom_ops.dq_summary shows your planted bad rows' rules with non-zero failures.

► SQL editor

```
-- explore the event log (use your pipeline id)
SELECT timestamp, event_type, message
FROM event_log('[pipeline_id]') ORDER BY timestamp DESC LIMIT 50;

SELECT * FROM portfolio.ecom_ops.run_checks ORDER BY checked_at DESC;
SELECT * FROM portfolio.ecom_ops.dq_summary ORDER BY run_at DESC, failed_pct DESC;
```

Create job ecom_daily_job (UI)

Setting	Value
Name	ecom_daily_job
Task 1	run_pipeline · type Pipeline · ecom_lakehouse_pipeline · full refresh off
Task 2	verify_run · Notebook 02_verify_pipeline_run · depends on run_pipeline
Task 3	dq_report · Notebook 03_data_quality_report · depends on run_pipeline · parameter pipeline_id
Job parameters	run_id = {{job.run_id}}, pipeline_id = [pipeline_id]
Trigger	File arrival on /Volumes/portfolio/ecom_landing/raw/, minimum time between triggers 60 seconds
Notifications	Email on failure (and on success while you demo)
Retries	run_pipeline: 1 retry after 5 minutes
Tags	project = ecom, env = dev

► Terminal · repo root

```
databricks jobs list | grep ecom_daily_job          # note job_id
databricks jobs run-now [job_id]                    # manual run
python scripts/upload_batch.py --next                # or let the file-arrival trigger start it
databricks jobs list-runs --job-id [job_id] --limit 5
```

► Terminal · repo root

```
git add -A && git commit -m "feat: orchestration and data quality report" && git push
```

B9. Day 12: governance

Put these statements in `sql/tags_and_comments.sql` and `sql/grants.sql` (versioned) and run them in the SQL editor. Repeat the comment and tag pattern for every gold table.

► SQL editor · `sql/tags_and_comments.sql`

```
COMMENT ON TABLE portfolio.ecom_gold.agg_daily_sales IS
'Daily delivered revenue, orders and late-delivery rate. Refresh: every run.';
ALTER TABLE portfolio.ecom_gold.agg_daily_sales
  ALTER COLUMN revenue COMMENT 'Sum of item price, BRL, delivered orders';

ALTER TABLE portfolio.ecom_gold.agg_daily_sales SET TAGS ('layer' = 'gold', 'domain' = 'sales');
ALTER TABLE portfolio.ecom_gold.dim_customer ALTER COLUMN city SET TAGS ('pii' = 'true');
ALTER TABLE portfolio.ecom_gold.dim_customer ALTER COLUMN zip_code_prefix SET TAGS ('pii' =
'true');
```

► SQL editor · `sql/grants.sql`

```
-- if you can create groups (Settings → Identity and access → Groups): ecom_analysts,
-- ecom_engineers
GRANT USE CATALOG ON CATALOG portfolio TO `ecom_analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA portfolio.ecom_gold TO `ecom_analysts`;
GRANT USE CATALOG ON CATALOG portfolio TO `ecom_engineers`;
GRANT ALL PRIVILEGES ON SCHEMA portfolio.ecom_bronze TO `ecom_engineers`;
GRANT ALL PRIVILEGES ON SCHEMA portfolio.ecom_silver TO `ecom_engineers`;
GRANT ALL PRIVILEGES ON SCHEMA portfolio.ecom_gold TO `ecom_engineers`;

SHOW GRANTS ON SCHEMA portfolio.ecom_gold;
```

WATCH OUT

Tables created by a pipeline are managed by it, so some ALTER statements may be refused on them. If so, set table and column comments in the pipeline code (the table declaration's comment and schema), and add tags in Catalog Explorer → table → Tags. Keep the SQL file for tables you manage yourself (ecom_ops).

If Free Edition doesn't let you create groups, keep `grants.sql` in the repo, explain the intended access model in `docs/governance.md`, and note it in the README. Then open Catalog Explorer → `ecom_gold.agg_daily_sales` → **Lineage** → **See lineage graph** and screenshot it.

► Terminal · repo root

```
git add -A && git commit -m "docs: governance and lineage" && git push
```

B10. Day 13: dashboard

- 1 Browser: **Dashboards** → **Create dashboard**. Name E-commerce Sales Overview.
- 2 **Data** tab: add a dataset per widget by selecting a gold table or writing a SQL query (datasets below).
- 3 **Canvas**: add the visuals; add global filters for date range and state; create 4 pages.
- 4 Publish, then take one full-page screenshot per page into `docs/screenshots/`.

Page	Widget	Dataset	Visual
Sales	Revenue, orders, AOV counters	<code>agg_daily_sales</code>	3 counters
Sales	Revenue by month	<code>agg_daily_sales</code> grouped by month	Line
Sales	Top 10 categories	<code>agg_category_monthly</code>	Bar
Sales	Revenue by state	<code>fact_order_item</code> join <code>dim_customer</code> where <code>__END_AT</code> is null	Bar or map
Operations	Late delivery rate by month	<code>agg_daily_sales</code>	Line
Operations	Seller leaderboard	<code>agg_seller_performance</code>	Table

Page	Widget	Dataset	Visual
Operations	Review score by delivery days band	fact_order_item	Bar
Customers	Cohort retention	agg_customer_cohorts	Heatmap
Data quality	Failure % per rule and run	ecom_ops.dq_summary	Table + bar

► Terminal · repo root

```
git add -A && git commit -m "feat: sales dashboard" && git push
```

B11. Day 14: deploy with an Asset Bundle

Generate bundle files from the pipeline and job you built in the UI, then parameterize them.

► Terminal · repo root

```
# optional: look at a reference layout
databricks bundle init default-python --output-dir /tmp/bundle-example

# generate YAML from the existing resources (writes into resources/ and src/)
databricks bundle generate pipeline --existing-pipeline-id [pipeline_id] --key \
  ecom_lakehouse_pipeline
databricks bundle generate job --existing-job-id [job_id] --key ecom_daily_job
```

File	What to set (YAML settings)
databricks.yml	bundle.name = ecom_lakehouse · include resources/*.yml · variables: catalog (default portfolio), landing_path (default /Volumes/portfolio/ecom_landing/raw), alert_email · targets: dev (mode development, default true, workspace.host = your URL)
resources/ecom_lakehouse_pipeline.yml	Replace hard-coded catalog and path with \${var.catalog} and \${var.landing_path}; libraries point to ../src/ecom/pipelines/*.py; serverless true
resources/ecom_daily_job.yml	Pipeline task references \${resources.pipelines.ecom_lakehouse_pipeline.id} (not a hard-coded id); notebook paths relative to the repo; email_notifications on_failure = \${var.alert_email}

► Terminal · repo root

```
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle summary -t dev # shows deployed resource names and URLs
databricks bundle run -t dev ecom_daily_job # run the job
# rebuild all tables
databricks bundle run
-t dev ecom_lakehouse_pipeline --full-refresh-all
```

Delete the UI-created pipeline and job (they are replaced by the bundle's [dev your-name]-prefixed copies), run the bundle job, and confirm everything works. Add a “Deploy” section with these commands to the README.

► Terminal · repo root

```
git add -A && git commit -m "feat: asset bundle deployment" && git push
```

B12. Day 15: full clean run-through

► SQL editor

```
-- clean start (careful: deletes all project tables)
DROP SCHEMA IF EXISTS portfolio.ecom_bronze CASCADE;
DROP SCHEMA IF EXISTS portfolio.ecom_silver CASCADE;
DROP SCHEMA IF EXISTS portfolio.ecom_gold CASCADE;
DROP SCHEMA IF EXISTS portfolio.ecom_ops CASCADE;
CREATE SCHEMA portfolio.ecom_bronze; CREATE SCHEMA portfolio.ecom_silver;
CREATE SCHEMA portfolio.ecom_gold; CREATE SCHEMA portfolio.ecom_ops;
```

► Terminal · repo root

```
# empty the landing Volume
databricks fs rm -r dbfs:/Volumes/portfolio/ecom_landing/raw/
rm -f data/upload_log.csv
databricks bundle deploy -t dev
python scripts/upload_batch.py --batch 0000_initial
python scripts/upload_batch.py --all # file-arrival trigger processes the drops
python scripts/upload_batch.py --batch 9999_bad_rows
```

Then record for the README: rows per layer, average run time, % quarantined per table, number of SCD2 versions, and all checks passing in `ecom_ops.run_checks`.

B13. Azure track: ADLS Gen2 landing, Key Vault and Azure SQL serving

Goal: show the same lakehouse running the way an Azure client runs it: files land in **ADLS Gen2**, Databricks reads them through Unity Catalog with a managed identity (no keys), and curated results are published to **Azure SQL Database** for an application. Prerequisite: A9 done.

Component	Free Edition version	Azure track version
Landing	Managed Volume portfolio.ecom_landing.raw	External Volume with the same name on abfss://landing@[ST]/ecom (pipeline code unchanged)
Upload	upload_batch.py → Volume	upload_batch.py --target adls (Data Lake SDK) or azcopy
Credentials	Databricks token	Managed identity (Access Connector) for storage; Key Vault scope for SQL
Serving	Dashboard only	Dashboard + Azure SQL servingdb tables for apps + optional Power BI
Deployment	Bundle target dev	Bundle target azure (host = Azure Databricks URL)

Step 1: point the landing Volume at ADLS

► SQL editor · Azure Databricks

```
CREATE EXTERNAL VOLUME IF NOT EXISTS portfolio.ecom_landing.raw
LOCATION 'abfss://landing@stdbxportfolio[SUFFIX].dfs.core.windows.net/ecom';
```

Because the Volume keeps its name, `/Volumes/portfolio/ecom_landing/raw` still works in the pipeline; only the storage behind it changed. That's a design point worth a paragraph in the README.

Step 2: extend upload_batch.py with an ADLS target

scripts/upload_batch.py (Azure option)

New arguments	--target volume adls (default volume) --account [storage account] --container landing --prefix ecom
Logic	1. When target is adls: create a DataLakeServiceClient for https://[account].dfs.core.windows.net using DefaultAzureCredential (your az login; no keys). 2. Get the file system client for the container; for each file build the path [prefix]/[source]/[file name]. 3. Skip if the path already exists (get file properties; a not-found error means upload). 4. Create the directory if needed, upload with overwrite disabled, retry 3 times on transient errors. 5. Log exactly as the Volume target does, with target = adls in the log line.
Done when	Files appear under landing/ecom/[source]/ and a second upload skips them.

► Terminal · repo root

```

pip install azure-identity azure-storage-file-datalake
python scripts/upload_batch.py --target adls --account $ST --batch 0000_initial
python scripts/upload_batch.py --target adls --account $ST --next
# bulk alternative:
azcopy login
azcopy copy "data/batches/2017_01/*" "https://$ST.dfs.core.windows.net/landing/ecom/" \
  --recursive
az storage fs file list -f landing --path ecom --account-name $ST --auth-mode login -o table \
  | head

```

TIP

For high file volumes, turn on **file events** on the loc_landing external location (Catalog Explorer → the location → settings) so Auto Loader uses storage notifications instead of listing directories. Mention it in the README.

Step 3: serving database in Azure SQL

► Terminal · repo root

```

az sql server create -n $SQLSRV -g $RG -l $LOC --admin-user sqladminuser --admin-password \
  '[Strong-Passw0rd!]'
az sql server firewall-rule create -g $RG -s $SQLSRV -n allow-azure --start-ip-address \
  0.0.0.0 --end-ip-address 0.0.0.0
az sql server firewall-rule create -g $RG -s $SQLSRV -n allow-my-ip \
  --start-ip-address $(curl -s https://api.ipify.org) --end-ip-address $(curl -s \
  https://api.ipify.org)
az sql db create -g $RG -s $SQLSRV -n servingdb --edition GeneralPurpose --compute-model \
  Serverless \
  --family Gen5 --capacity 1 --auto-pause-delay 60 --use-free-limit \
  --free-limit-exhaustion-behavior AutoPause

az keyvault secret set --vault-name $KV -n sql-user --value sqladminuser
az keyvault secret set --vault-name $KV -n sql-password --value '[Strong-Passw0rd!]'
az keyvault secret set --vault-name $KV -n sql-host --value $SQLSRV.database.windows.net

```

notebooks/04_publish_to_azure_sql

Purpose	Publish curated gold tables to Azure SQL so an application can query them.
Parameters	tables (default agg_daily_sales, agg_seller_performance), database servingdb

Logic	1. Read host, user and password from the kv-portfolio secret scope (never print them). 2. Build the JDBC URL jdbc:sqlserver://[host]:1433;database=servingdb;encrypt=true;trustServerCertificate=false;loginTimeout=30. 3. For each table: read the gold table; cast decimals to decimal(12,2) and rates to decimal(9,4) so SQL types are predictable. 4. Write to dbo.[table] with mode overwrite and the truncate option on (keeps the SQL table, its indexes and permissions), batch size 10,000. 5. Read back SELECT COUNT(*) through JDBC and compare with the gold count; fail the task if they differ. 6. Append (table, rows, published_at) to ecom_ops.publish_log.
Done when	sqlcmd shows the same row counts as the gold tables.

► Terminal · repo root

```
sqlcmd -S $SQLSRV.database.windows.net -d servingdb -U sqladminuser -P '[Strong-Passw0rd!]' \
-Q "SELECT COUNT(*) FROM dbo.agg_daily_sales; SELECT TOP 5 * FROM \
    dbo.agg_seller_performance ORDER BY revenue DESC;"
```

Step 4: deploy to the Azure workspace

- 1 Add a target azure to databricks.yml: mode development, workspace host = your Azure Databricks URL.
- 2 Add a task publish_to_sql (notebook 04_publish_to_azure_sql) to ecom_daily_job, depending on verify_run, and point it at a small job cluster (single node, Standard_D4ds_v5).
- 3 Deploy, upload batches, and watch the file-arrival trigger run the whole chain.
- 4 Optional: Power BI Desktop → Get data → Azure Databricks (SQL warehouse connection details) → build one page on the gold tables.

► Terminal · repo root

```
databricks bundle validate -t azure --profile azure
databricks bundle deploy -t azure --profile azure
databricks bundle run -t azure ecom_daily_job --profile azure
```

Screenshots for the README (Azure section)

- ☐ Storage account containers in the portal
- ☐ External location and credential in Catalog Explorer
- ☐ Job run including publish_to_sql
- ☐ Query results from servingdb
- ☐ Resource group cost after the session

Project 1 acceptance checklist

- ☐ 10 bronze, 10 silver + quarantine, 10 gold tables from one pipeline
- ☐ Incremental runs proven
- ☐ Rules and quarantine proven with planted bad rows
- ☐ SCD2 history visible
- ☐ Job: trigger, verify, DQ tasks
- ☐ Comments, tags, grants, lineage
- ☐ 4-page dashboard
- ☐ Tests passing
- ☐ Bundle deploys from scratch
- ☐ README, diagrams, blog post

PART F

Finishing, Publishing and Troubleshooting

F1. README structure

Section	Content
Title + badges	Name; CI badge; tech badges
Business problem	Client story, 2-3 sentences
Results	3-5 bullets with measured numbers
Architecture	Diagram + walkthrough
Data model	Table catalogue and diagram
Design decisions	Choices and alternatives
Screenshots	Graph, lineage, dashboard, CI, alerts
How to run	Every command from this manual, in order
Testing	How to run tests; coverage
Limitations and next steps	Honest list

F2. Publish each project

► Terminal · repo root

```
gh repo edit --description "Databricks lakehouse: Auto Loader, Lakeflow pipelines, SCD2, \
  Unity Catalog, AI/BI" \
  --add-topic databricks --add-topic pyspark --add-topic delta-lake --add-topic \
  data-engineering
git tag -a v1.0 -m "Portfolio release" && git push origin v1.0
gh release create v1.0 --title "v1.0" --notes "First complete version"
```

- 1 GitHub profile → Customize your pins → select the repo.
- 2 Publish the blog post; post on LinkedIn (hook with the result, 4-5 points, diagram; repo link in first comment).
- 3 Add to the Upwork portfolio and the LinkedIn Featured section.

F3. Troubleshooting

Problem	Cause	Fix / command
CLI: "cannot configure default credentials"	No profile selected	export DATABRICKS_CONFIG_PROFILE=portfolio, or add --profile portfolio
Pipeline: ModuleNotFoundError ecom	src not on the path	Set the pipeline root folder to the repo's src, or add the path in the pipeline source settings
Review rows broken across lines	Multi-line text	multiLine plus quote and escape options for order_reviews
Silver drops nearly everything	Rule checked before cleaning, or case mismatch	Apply rules after type_ functions; unit-test the rules
Re-uploaded file ignored	Auto Loader skips processed files	New files need new names; landing is immutable
Need to rebuild everything	Logic changed for existing data	databricks bundle run -t dev [pipeline] --full-refresh-all

Problem	Cause	Fix / command
SCD2 versions out of order	Wrong or non-unique sequence	Sequence by change_ts / seq; break ties with the key
pytest: Java gateway exited	Java missing or JAVA_HOME wrong	java -version; set JAVA_HOME (A3)
CI: bundle validate auth error	Secret names differ	gh secret list; names must match the workflow exactly
psql: SSL required	Neon needs SSL	Keep sslmode=require in PG_CONNECTION
Free Edition quota reached	Heavy compute in one day	Fewer TLC months; run experiments locally; continue next day
Notebook can't reach internet/Postgres	Outbound access restricted	Run on the laptop and upload to a Volume
Azure: 403 AuthorizationPermissionMismatch on ADLS	Control-plane role instead of data role, or role not propagated yet	Assign Storage Blob Data Contributor; wait up to 10 minutes; check the external location's credential
Azure Databricks: cluster fails with QuotaExceeded	vCPU quota for the VM family is too low	az vm list-usage -l \$LOC -o table; use a smaller single-node size or request more quota in the portal (Quotas)
Azure SQL: login failed / client IP not allowed	Firewall rule missing	az sql server firewall-rule create ...; for serverless Databricks compute configure network access
Azure SQL: CREATE USER FROM EXTERNAL PROVIDER fails	Not connected as the Entra admin	sqlcmd -G as the Entra admin set with az sql server ad-admin create
Cosmos DB: 429 Too Many Requests	Writes exceed provisioned RU/s	Enable bulk and throughput control; raise RU/s temporarily
Function App: runs but writes nothing	Managed identity lacks the data role, or wrong app settings	az functionapp identity show; check role assignments and az functionapp config appsettings list
Secret scope: cannot read Key Vault	AzureDatabricks app lacks Key Vault Secrets User	Assign the role to app id 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d
Unexpected Azure bill	Clusters or VM left running	Auto-termination, az vm deallocate, delete the resource group after each track
Benchmark times vary widely	Warm-up and shared resources	3 runs, median, drop outputs before each run

PART G

Command Reference

Every command used in this manual, grouped by tool, for quick lookup.

Git

► Terminal

```
git clone git@github.com:[user]/[repo].git
git status
git pull
git checkout -b feature/[name]
git add -A
git commit -m "type: summary"
git push -u origin feature/[name]
git checkout main
git branch -d feature/[name]
git log --oneline --graph -15
git diff
git restore [file]
git tag -a v1.0 -m "Portfolio release"
git push origin v1.0
```

Python environment and quality

► Terminal

```
python3.11 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
pip freeze > requirements-lock.txt
ruff check .
ruff check . --fix
ruff format .
pytest -q
pytest -q -k [test_name]
pytest -q --cov=src --cov-report=term-missing
```

Databricks CLI: auth, files, pipelines, jobs

► Terminal

```
databricks auth login --host https://[workspace-url] --profile portfolio
databricks auth profiles
databricks current-user me
databricks catalogs list
databricks schemas list portfolio
databricks volumes list portfolio [schema]
databricks fs ls dbfs:/Volumes/portfolio/[schema]/[volume]/
databricks fs cp [local-file] dbfs:/Volumes/portfolio/[schema]/[volume]/[file]
databricks fs cp -r [local-folder] dbfs:/Volumes/portfolio/[schema]/[volume]/[folder]
databricks fs cp --overwrite [local-file] dbfs:/Volumes/.../[file]
databricks fs rm -r dbfs:/Volumes/portfolio/[schema]/[volume]/[folder]
databricks pipelines list-pipelines
databricks pipelines start-update [pipeline_id]
databricks pipelines get [pipeline_id]
databricks pipelines stop [pipeline_id]
databricks jobs list
databricks jobs run-now [job_id]
databricks jobs list-runs --job-id [job_id] --limit 5
```

Databricks Asset Bundles

► Terminal

```
databricks bundle init # start from a template
databricks bundle generate pipeline --existing-pipeline-id [id] --key [name]
databricks bundle generate job --existing-job-id [id] --key [name]
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev [job_or_pipeline_key]
databricks bundle run -t dev [pipeline_key] --full-refresh-all # rebuild all tables
databricks bundle summary -t dev
databricks bundle destroy -t dev # removes deployed resources
```

SQL editor: inspection and maintenance

► SQL editor

```
SHOW CATALOGS;
SHOW SCHEMAS IN portfolio;
SHOW TABLES IN portfolio.[schema];
SHOW VOLUMES IN portfolio.[schema];
DESCRIBE TABLE EXTENDED portfolio.[schema].[table];
DESCRIBE DETAIL portfolio.[schema].[table];
DESCRIBE HISTORY portfolio.[schema].[table];
SELECT * FROM portfolio.[schema].[table] VERSION AS OF [n];
OPTIMIZE portfolio.[schema].[table];
ALTER TABLE portfolio.[schema].[table] CLUSTER BY ([col1], [col2]);
COMMENT ON TABLE portfolio.[schema].[table] IS '[text]';
ALTER TABLE portfolio.[schema].[table] SET TAGS ('[key]' = '[value]');
GRANT SELECT ON SCHEMA portfolio.[schema] TO `[group]`;
SHOW GRANTS ON SCHEMA portfolio.[schema];
SELECT * FROM event_log('[pipeline_id]') ORDER BY timestamp DESC LIMIT 50;
```

Data download and PostgreSQL

► Terminal

```
kaggle datasets download -d olistbr/brazilian-ecommerce -p data/raw --unzip
curl -L -o [file] \
  https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_[YYYY]-[MM].parquet
docker run --name shopdb -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=shopdb -p 5432:5432 -d \
  postgres:16
docker start shopdb
docker stop shopdb
docker logs shopdb
set -a && source .env && set +a
psql "$PG_CONNECTION" -f source_db/schema.sql
psql "$PG_CONNECTION" -c "\dt"
```

Project scripts

► Terminal

```
# Project 1
python scripts/prepare_batches.py --raw data/raw --out data/batches [--months N] [--force]
python scripts/generate_customer_updates.py --batch YYYY_MM --pct 2 --seed 42
python scripts/make_bad_rows_batch.py --from-batch 2018_08 --out-batch 9999_bad_rows
python scripts/upload_batch.py --batch [id] | --next | --all [--dry-run]

# Project 2
python scripts/download_tlc.py --year 2023 --months 1-12 --out data/tlc
python scripts/upload_tlc.py
python local/skew_experiment.py --rows 20000000 --hot-share 0.6
python local/shuffle_partitions_experiment.py --rows 10000000

# Project 3
python simulator/seed_source.py --customers 5000 --orders 20000 --seed 7 [--reset]
python simulator/generate_changes.py --duration-min 10 --rate 5 --mix 50,40,10
python extractor/extract_changes.py --env dev|prod [--full] [--dry-run]
```

Azure CLI

► Terminal

```
az login
az account set --subscription "[name or id]"
az group create -n $RG -l $LOC
az group delete -n $RG --yes --no-wait
az storage account create -n $ST -g $RG --kind StorageV2 --hns true --sku Standard_LRS
az storage fs create -n [container] --account-name $ST --auth-mode login
az storage fs file list -f [container] --path [folder] --account-name $ST --auth-mode login \
-o table
az role assignment create --assignee-object-id [id] --assignee-principal-type \
[User|ServicePrincipal] --role "[role]" --scope [id]
az role assignment list -g $RG -o table
az keyvault create -n $KV -g $RG --enable-rbac-authorization true
az keyvault secret set --vault-name $KV -n [name] --value [value]
az databricks workspace create -n dbw-portfolio -g $RG --sku trial
az databricks access-connector create -n ac-dbx-portfolio -g $RG --identity-type SystemAssigned
az sql server create -n $SQLSRV -g $RG --admin-user [user] --admin-password [password]
az sql server ad-admin create -g $RG -s $SQLSRV --display-name [name] --object-id $ME
az sql server firewall-rule create -g $RG -s $SQLSRV -n [rule] --start-ip-address [ip] \
--end-ip-address [ip]
az sql db create -g $RG -s $SQLSRV -n [db] --edition GeneralPurpose --compute-model \
Serverless --family Gen5 \
--capacity 1 --use-free-limit --free-limit-exhaustion-behavior AutoPause
az cosmosdb create -n $COSMOS -g $RG --enable-free-tier true
az cosmosdb sql container create -a $COSMOS -g $RG -d [db] -n [container] \
--partition-key-path "/"[key]" --throughput 400
az functionapp create -g $RG -n $FUNC --storage-account $ST --flexconsumption-location $LOC \
--runtime python --runtime-version 3.11
az functionapp config appsettings set -g $RG -n $FUNC --settings KEY=value
az vm create -g $RG -n [vm] --image [image] --size [size] --admin-username [user]
az vm deallocate -g $RG -n [vm]
az vm list-usage -l $LOC -o table
az deployment group what-if -g $RG -f infra/main.bicep -p infra/parameters.dev.json
az deployment group create -g $RG -f infra/main.bicep -p infra/parameters.dev.json
```

Azure data tools

► Terminal

```
azcopy login
azcopy copy "[local folder]/*" "https://$ST.dfs.core.windows.net/[container]/[path]/" \
--recursive
sqlcmd -S $SQLSRV.database.windows.net -d [db] -G -Q "[query]" # Entra auth
sqlcmd -S $SQLSRV.database.windows.net -d [db] -U [user] -P [password] -i [file.sql]
func init [name] --python
func new --name [function] --template "Timer trigger"
func start
func azure functionapp publish $FUNC
```

GitHub CLI

► Terminal

```
gh auth login
gh repo edit --description "[text]" --add-topic [topic]
gh secret set [NAME]
gh secret set [NAME] --env production
gh secret list
gh pr create --fill
gh pr checks --watch
gh pr merge --squash --delete-branch
gh run list --limit 5
gh run view [run_id] --log-failed
gh release create v1.0 --notes "[text]"
```

Remember: finished and documented beats perfect. Publish Project 1 as soon as its checklist is done, start sending proposals, and build Projects 2 and 3 while you look for clients.